COURSE SLIDES

Multi Agents System with AutoGen (C1034)

Course Code: C1034

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v1.0 · 11 August 2026

© 2026 Tertiary Infotech Academy Pte Ltd. All rights reserved. · www.tertiarycourses.com.sg

COURSE ADMINISTRATION

Welcome & Housekeeping

© 2026 Tertiary Infotech Academy Pte Ltd

About the Trainer



Your Trainer

General Trainer template —
to be completed by the trainer

NAME

TITLE / DESIGNATION

QUALIFICATIONS

AREAS OF EXPERTISE

TRAINING & INDUSTRY EXPERIENCE

CONTACT

© 2026 Tertiary Infotech Academy Pte Ltd

YOUR TRAINER

About the Trainer



Tertiary Infotech Trainer

Principal Trainer
Tertiary Infotech Academy Pte. Ltd.

ROLE
Principal Trainer, Tertiary Infotech Academy Pte. Ltd.

CERTIFICATION
Applied AI and multi-agent systems practitioner

DELIVERS
AutoGen AgentChat, RAG, multi-agent workflow design, safe tool use, verification, and operational controls

FOUNDER
Founder and lead instructor at Tertiary Infotech / Tertiary Courses.

© 2026 Tertiary Infotech Academy Pte Ltd

ICE-BREAKER

Let's Know Each Other

1. Your name and organisation / role.

2. Your experience with this subject (if any).

3. What you want to be able to do after this course.

© 2026 Tertiary Infotech Academy Pte Ltd

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. Learn by doing — try every hands-on lab.

© 2026 Tertiary Infotech Academy Pte Ltd

COURSE PORTAL

Download Course Material

1. Sign in at the LMS-TMS portal
2. Open this course from your dashboard
3. Go to the Courseware tab
4. Download the Slides (PPT/PDF), Learner Guide and Lesson Plan
5. Keep them open alongside the labs as you work

Portal: https://lms-tms.tertiaryinfotech.com

© 2026 Tertiary Infotech Academy Pte Ltd

SCHEDULE

Lesson Plan — 2 Days, 8 scheduled hours/day

Day 1

- Day 1 — AgentChat Foundations and Grounded Retrieval
  - Topic 1: Introduction to AutoGen and CrewAI (Labs 1–2)
  - Topic 2: Retrieval-Augmented Generation (RAG) (Labs 3–5)

Day 2

- Day 2 — Multi-Agent Orchestration and Operational Readiness
  - Topic 3: Implementing a Multi-Agent AI Workflow (Labs 6–8)
- Daily timing
  - 9:30 am - 6:30 pm (1-hour lunch; two 15-minute tea breaks)

© 2026 Tertiary Infotech Academy Pte Ltd

WHAT YOU'LL ACHIEVE

Learning Outcomes

1

Agent Foundations

Explain LLM agents, the AgentChat execution model, and the practical differences between AutoGen and CrewAI abstractions.

2

AgentChat in Practice

Configure and run an AutoGen AssistantAgent with a model client, clear messages, a typed read-only tool, and inspectable event evidence.

3

Retrieval Design

Design a Retrieval-Augmented Generation pipeline with source-labelled chunks, transparent ranking, metadata, context limits, and stable retrieval questions.

4

Grounded RAG

Build and verify a grounded RAG agent that cites retrieved evidence, treats documents as untrusted data, and handles insufficient evidence honestly.

5

Team Orchestration

Select and implement appropriate multi-agent patterns with specialist roles, explicit handoffs, shared context, and bounded speaker routing.

6

Operational Control

Apply termination, state management, guardrails, observability, regression checks, ownership, and recovery controls to an integrated AutoGen workflow.

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING APPROACH

How You'll Learn

1. Trainer demonstrates the concept
2. You build it yourself in the hands-on lab
3. Verify with the lab's 'Test it' step
4. Discuss and troubleshoot together
5. Recap against the learning outcomes

© 2026 Tertiary Infotech Academy Pte Ltd

CORE CONCEPTS

Course Fundamentals

© 2026 Tertiary Infotech Academy Pte Ltd

OVERVIEW

The AgentChat System Model

1

Task

A new user request or message list that starts or resumes a run.

2

Agent

A named, stateful component with instructions, a model client and optional tools.

3

Model client

The provider adapter that creates chat completions and reports usage.

4

Tool

A typed application capability executed outside the model and returned as an event.

5

Message

Conversation content from a user, agent or control component.

6

Event

Inspectable evidence such as a tool request, result or streaming chunk.

© 2026 Tertiary Infotech Academy Pte Ltd

The AgentChat System Model (cont.)

7

Team

A stateful group that selects speakers and applies termination conditions.

8

Memory

A component that retrieves relevant information into model context.

9

Termination

The condition and stop reason that bound a run.

10

TaskResult

The returned messages plus the reason the run stopped.

© 2026 Tertiary Infotech Academy Pte Ltd

The GROUND Design Lens

1

G — Goal

Define the user outcome, non-goals and acceptable failure behaviour.

2

R — Roles

Give agents and tools distinct responsibilities and least privilege.

3

O — Organise

Choose retrieval, routing and handoff patterns that fit the work.

4

U — Use evidence

Preserve source identifiers and validate claims against retrieved content.

5

N — Name limits

Set completion, message, time, token and tool budgets.

6

D — Diagnose

Record routes, events, stop reasons, errors, quality and recovery evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

ARCHITECTURE PRINCIPLE

More agents create more interfaces—not automatically more intelligence.

Add a participant only when the boundary improves specialisation, permission control, ownership or verification enough to justify extra routing, cost and failure modes.

© 2026 Tertiary Infotech Academy Pte Ltd

What You Will Build

Agent Core

- Replay and optional live model clients
- A typed read-only ticket tool
- Inspectible messages and events

Grounded Knowledge

- Source-labelled handbook chunks
- Stable retrieval questions
- A cited RAG answer contract

Bounded Team

- Round-robin review
- Selector-based specialist routing
- Termination, regression and recovery evidence

© 2026 Tertiary Infotech Academy Pte Ltd

THE LEARNING ARC

How Every Lab Progresses

1. Understand the concept, decision and failure mode before changing code.
2. Restore or verify the prior lab checkpoint so the shared scenario is stable.
3. Make one bounded change using synthetic data and placeholder secrets.
4. Run an observable command and compare its evidence with the expected result.
5. Keep the checkpoint and reflection that prepare the next lab.

© 2026 Tertiary Infotech Academy Pte Ltd

01

TOPIC 01

Introduction to AutoGen and CrewAI

LLMs and agents · AutoGen layers · AgentChat lifecycle · AssistantAgent · tools · messages · state · framework choice

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Introduction to AutoGen and CrewAI

1

LLM, agent and multi-agent system

An LLM predicts outputs from context. An agent wraps a model with instructions, tools, state and an execution loop. A multi-agent system adds role boundaries and communication, which increases coordination work as well as capability.

2

The agent loop

A task becomes messages; the model chooses a response or tool call; the application executes approved tools; the result returns as another event; and an explicit rule decides whether to continue or stop.

3

AutoGen layers

Studio supports visual prototyping, AgentChat provides high-level conversational agents and teams, Core exposes event-driven primitives, and Extensions connect model clients, memory, runtimes and tools.

4

AgentChat contract

AgentChat agents have unique names and useful descriptions, accept tasks through run or run_stream, emit typed messages and events, and retain state until reset. The returned TaskResult contains the run history and stop reason.

5

AssistantAgent

AssistantAgent is a general preset for prototyping and education. Its contract combines a model client, system message, optional tools, model context and bounded tool iterations; specialised production agents may use narrower custom implementations.

6

Messages and events

User and agent text are chat messages. Tool requests, tool results and streaming chunks are events. Operators examine both because a fluent final answer cannot prove that the correct route or evidence was used.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Introduction to AutoGen and CrewAI (cont.)

7

Model clients

A model client separates orchestration from a provider. The course uses ReplayChatCompletionClient for reproducible offline evidence and an OpenAI client only for optional live comparison through environment-held credentials.

8

Typed tools

A function name, docstring, parameter types and return shape become the tool schema. Clear read-only contracts reduce ambiguous calls and make argument validation, error handling and evidence capture testable.

9

State and reset

Agents and teams retain conversation state after a run. Continue only when the next task belongs to the same thread; reset before an unrelated task so stale context does not silently influence a result.

10

Bounded autonomy

Every loop needs a useful completion condition plus a hard message, tool-iteration, token or time limit. A limit is not a success signal; the stop reason must remain visible to the application.

11

AutoGen and CrewAI

AutoGen AgentChat centres on typed messages, events, model clients and team presets. CrewAI organises role-based agents into crews and tasks, while Flows provide explicit event-driven control. Choose by required control, integrations and team operating model.

12

Worked example: OpsDesk

A synthetic service request enters one read-only assistant. The agent looks up an incident, later retrieves handbook evidence, then grows into planner, retriever, writer and reviewer roles without gaining permission to change production systems.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Verify AutoGen and Run a First Agent

LAB 1

You prepare an isolated Python environment, verify the pinned AutoGen AgentChat and Extensions packages, and run the first OpsDesk AssistantAgent with a ReplayChatCompletionClient. You then inspect the saved message types, final text and stop reason so the execution model is observable before any live model or tool is introduced.

You'll build

A ready environment plus .course_state/lab01-environment.json and lab01-first-agent.json containing the first reproducible AgentChat trace.

Tools: Python 3.10+, virtual environment, AutoGen AgentChat 0.7.5, AutoGen Extensions replay client, OpsDesk CLI

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 1 — Key Steps

1. Create and activate an isolated Python environment.
2. Install the pinned lab dependencies and editable OpsDesk package.
3. Generate the environment report and confirm status ready.
4. Run the first AgentChat assistant with the replay client.
5. Inspect messages, final content and stop reason; keep the checkpoint.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 1 · VERIFY

Verify AutoGen and Run a First Agent

Test it

The setup report says ready and the first trace contains user and assistant messages with the expected synthetic-data boundary.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 01 · HANDS-ON

Add a Typed Ticket Lookup Tool

LAB 2

You extend the OpsDesk assistant with lookup_ticket, a typed function that reads one synthetic incident by exact identifier. The offline replay client produces a real function-call request, AgentChat executes the tool, and the saved trace proves the request, result and not-found behaviour without changing any ticket.

You'll build

A verified lookup_ticket contract plus .course_state/lab02-tool-agent.json containing ToolCallRequestEvent, ToolCallExecutionEvent and the exact synthetic result.

Tools: AssistantAgent, FunctionTool schema, FunctionCall, ToolCallExecutionEvent, ReplayChatCompletionClient, synthetic tickets.json

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · EXECUTION MAP

Lab 2 — Key Steps

1. Inspect the typed lookup_ticket signature and synthetic dataset.
2. Run an exact INC-1001 tool request through AssistantAgent.
3. Confirm request, execution and summary events in the trace.
4. Exercise an unknown identifier and verify not_found behaviour.
5. Run focused checks and retain the read-only checkpoint.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 2 · VERIFY

Add a Typed Ticket Lookup Tool

Test it

The trace contains a ToolCallExecutionEvent and returns the exact requested incident or an explicit not_found result without mutating tickets.json.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Introduction to AutoGen and CrewAI

1. You can now: LO1 and LO2: distinguish the AgentChat components, prepare a reproducible environment, run an AssistantAgent, and inspect messages and the TaskResult stop reason

2. You can now: LO2: define and execute a typed read-only tool, inspect its schema and tool events, and preserve explicit not-found behaviour without granting write authority

© 2026 Tertiary Infotech Academy Pte Ltd

02

TOPIC 02

Retrieval-Augmented Generation (RAG)

Knowledge preparation · chunking · lexical and vector retrieval · ranking · metadata · grounding · citations · retrieval evaluation

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Retrieval-Augmented Generation (RAG)

1

Why RAG exists

A model's parameters are not a current, inspectable knowledge store. RAG retrieves task-relevant evidence at run time and adds it to the model context so answers can be grounded in controlled sources.

2

The RAG pipeline

Collect and approve sources, split them into usable chunks, create a searchable representation, retrieve candidates, filter and rank them, augment the prompt, generate a response, and verify the cited evidence.

3

Source governance

Each chunk needs a stable source identifier, title, owner, version or date, data classification and access rule. Retrieval quality cannot compensate for stale, unauthorised or contradictory source material.

4

Chunk boundaries

Chunks should preserve one coherent rule or procedure. Very small chunks lose meaning; very large chunks waste context and can hide the relevant passage. Overlap is useful when a concept crosses a boundary but creates duplicates.

5

Lexical retrieval

Token or term matching is transparent, fast and effective for exact identifiers and distinctive policy language. It struggles with paraphrases and synonyms, which makes it useful as a teaching baseline and an operational fallback.

6

Vector retrieval

Embeddings map meaning into vectors so semantically related wording can be close even when terms differ. Distance is only a ranking signal; top results still require access filtering, metadata checks and evidence validation.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Retrieval-Augmented Generation (RAG) (cont.)

7

Top-k and thresholds

Top-k controls how many candidates enter the next stage. A minimum score can prevent weak matches from becoming context. Both settings trade recall against noise, latency and context cost and must be tuned on stable questions.

8

Hybrid and reranking

Hybrid retrieval combines lexical and semantic signals. A reranker can reorder candidates for the specific query. These layers improve difficult searches but add dependencies and must preserve source metadata.

9

Grounded answer contract

A grounded response separates claims from evidence, includes source identifiers, avoids unsupported additions and returns an explicit insufficient-evidence state when no approved passage supports the request.

10

Retrieved text is untrusted

Documents can contain mistakes or embedded instructions. The application treats retrieved passages as data, keeps system and policy instructions authoritative, filters access first and never executes instructions found inside a source.

11

Retrieval evaluation

Use stable questions with expected sources to measure whether relevant passages appear and where they rank. Examine miss cases before changing chunking, metadata, query formulation or ranking one variable at a time.

12

AutoGen memory options

AgentChat accepts memory components that update model context. AutoGen Extensions includes ChromaDB vector memory, while custom Memory implementations can support other stores. The same source, retrieval and answer contracts still apply.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Retrieval-Augmented Generation (RAG) (cont.)

13

Worked example

For a contractor VPN request, the transparent index should rank POL-VPN-01 first. The RAG agent may state manager approval, multi-factor enrolment and the end-date rule only when that source is present and cited.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Prepare a Source-Traceable Knowledge Index

LAB 3

You inspect the synthetic OpsDesk handbook, identify the stable policy and procedure boundaries, and convert each section into one transparent lexical chunk. The index keeps source ID, title, text and tokens together so every later ranking and answer can be traced back to the governing passage.

You'll build

.course_state/opsdesk-index.json and lab03-index-manifest.json with five coherent chunks and the expected POL-/PROC- source identifiers.

Tools: Markdown source, source metadata, Python tokenizer, lexical index, JSON manifest, OpsDesk knowledge module

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 3 — Key Steps

1. Inspect source ownership boundaries and stable identifiers.
2. Parse one coherent chunk per handbook section.
3. Build the transparent token index and manifest.
4. Verify count, identifiers, text and token preservation.
5. Keep the index checkpoint for stable retrieval questions.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 3 · VERIFY

Prepare a Source-Traceable Knowledge Index

Test it

The manifest lists exactly five source IDs in document order and every index record preserves its source ID, title, text and tokens.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Evaluate Retrieval with Stable Questions

LAB 4

You run three stable OpsDesk questions against the Lab 3 index and compare the top-ranked source with the expected policy or procedure. Because the teaching retriever exposes matched terms and scores, you can diagnose a miss without guessing whether chunking, query wording or ranking caused it.

You'll build

Three repeatable retrieval observations covering VPN approval, critical response time and embedded-instruction handling, with expected top-one source evidence.

Tools: Lexical ranking, inverse document frequency, top-k retrieval, stable questions, source IDs, pytest

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 4 — Key Steps

1. Define stable questions and their expected source identifiers.
2. Run VPN, response-time and embedded-instruction searches.
3. Inspect matched terms, scores and top-one source evidence.
4. Exercise an unrelated question and observe the evidence gap.
5. Run retrieval checks and keep the ranked evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 4 · VERIFY

Evaluate Retrieval with Stable Questions

Test it

All three stable questions rank the expected source first, while an unrelated question does not become a grounded answer.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 02 · HANDS-ON

Build a Grounded RAG Agent

LAB 5

You give a knowledge AssistantAgent access to the verified teaching retriever, require a tool call before answering and add a reflection step that turns tool evidence into a concise cited response. You then compare the supported VPN request with an unrelated question and confirm that retrieval events and source IDs remain visible.

You'll build

.course_state/lab05-rag-agent.json containing the RAG tool trajectory, grounded final text, source identifiers and stop reason for the shared VPN question.

Tools: AssistantAgent, retrieval FunctionTool, reflect_on_tool_use, max_tool_iterations, source citations, insufficient-evidence route

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 5 — Key Steps

1. Attach the verified retriever as a typed AgentChat tool.
2. Require retrieval before generation and bracketed source IDs.
3. Run the supported VPN question through the offline RAG agent.
4. Inspect tool events, grounded text, sources and stop reason.
5. Exercise insufficient evidence, run checks and restore the checkpoint.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 5 · VERIFY

Build a Grounded RAG Agent

Test it

The supported answer cites POL-VPN-01 after a tool execution event, while the unrelated request returns an insufficient-evidence statement without a fabricated source.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Retrieval-Augmented Generation (RAG)

1. You can now: LO3: split approved handbook content into coherent source-labelled chunks, build a transparent searchable index, and verify its manifest before retrieval

2. You can now: LO3: rank approved chunks for representative questions, compare top results with expected source IDs, and diagnose retrieval misses using visible terms and scores

3. You can now: LO4: connect retrieval as an AgentChat tool, require source-labelled responses, verify tool events and citations, and preserve an insufficient-evidence route

© 2026 Tertiary Infotech Academy Pte Ltd

03

TOPIC 03

Implementing a Multi-Agent AI Workflow

Specialist roles · round robin · selector routing · handoffs · termination · state · human control · observability · release readiness

© 2026 Tertiary Infotech Academy Pte Ltd

KEY CONCEPTS

Key Concepts — Implementing a Multi-Agent AI Workflow

1

When multiple agents help

Split work when roles need different instructions, tools, permissions, models, ownership or quality checks. Keep one agent when a single bounded instruction and tool surface remains understandable.

2

Role contract

Each participant needs a unique name, capability description, task boundary, approved tools, input evidence, output format and escalation behaviour. Overlapping roles create routing ambiguity and duplicate work.

3

Handoff contract

A handoff carries the task, validated context, source identifiers, constraints, expected output and remaining budget. It should not expose hidden reasoning, unrelated history or broader credentials.

4

RoundRobinGroupChat

Round robin gives every participant a predictable turn and shared context. It fits drafting and review loops or demonstrations where the order is intentional, but wastes turns when some roles are not needed.

5

SelectorGroupChat

A selector chooses the next speaker from names, descriptions and conversation context. It supports dynamic workflows but adds a routing decision that needs clear descriptions, candidate constraints, telemetry and fallback behaviour.

6

Custom selection

A selector function can enforce deterministic state transitions and return None to defer to model selection. Custom functions are useful for policy routes but require separate serialization and recovery design.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Implementing a Multi-Agent AI Workflow (cont.)

7

Other team patterns

Swarm uses handoff messages for decentralised transitions, Magentic-One targets open-ended generalist tasks, and GraphFlow expresses directed workflows. Pattern choice should follow control needs rather than novelty.

8

Termination

Semantic conditions such as an approval phrase or handoff should be combined with a hard maximum message, token or time condition. The team returns a stop reason that the application must interpret separately from answer content.

9

State lifecycle

Team and participant state persists after a run. Resume related work deliberately, reset unrelated work, and save or load state only at well-defined boundaries with the package and configuration version recorded.

10

Agent and team tools

Agents and teams can be wrapped as tools for composition, but they are stateful. Disable parallel calls when a model could invoke the same stateful participant concurrently and create inconsistent histories.

11

Human control

Pause for clarification, approval or high-impact actions. External termination can stop after the current turn, while application policy decides which requests require a person before any side effect.

12

Observability

Capture a correlation identifier, selected speaker, message and event types, tool name, source IDs, stop reason, duration, model usage and errors. Redact secrets and sensitive payloads before telemetry leaves the process.

© 2026 Tertiary Infotech Academy Pte Ltd

Key Concepts — Implementing a Multi-Agent AI Workflow (cont.)

13

Quality and safety checks

Test normal, missing-evidence, malformed, embedded-instruction, unsupported-action and forced-limit cases. Evaluate route and tool trajectory as well as the final response.

14

Cost and latency

Every selector and specialist turn can add model calls, tokens and delay. Set budgets, cache stable retrieval, use deterministic code where appropriate and compare the team against a one-agent baseline.

15

Worked example

OpsDesk first uses a writer-reviewer round robin, then a planner-retriever-writer-reviewer selector route. Both must cite POL-VPN-01, stop on approval or the hard limit, and leave the supplied tickets unchanged.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Design Roles and Run a Round-Robin Team

LAB 6

You split the Lab 5 response task into a writer that uses supplied evidence and a reviewer that checks support and citation. A RoundRobinGroupChat makes the order predictable, while the course's ExactTextTermination combined with MaxMessageTermination proves that an exact reviewer approval marker can end the normal run and a hard boundary still exists.

You'll build

.course_state/lab06-round-robin.json with participant names, observed speaker sequence, message count, stop reason and bounded status for the writer-reviewer workflow.

Tools: AssistantAgent roles, RoundRobinGroupChat, ExactTextTermination, MaxMessageTermination, shared context, TaskResult

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 6 — Key Steps

1. Define distinct writer and reviewer role contracts.
2. Create a fixed-order team with shared conversation context.
3. Combine approval termination with a six-message hard limit.
4. Run the VPN task and inspect speakers, count and stop reason.
5. Verify bounded behaviour and retain the team checkpoint.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 6 · VERIFY

Design Roles and Run a Round-Robin Team

Test it

The observed route is user to writer to reviewer, the reviewer approves supported content, and the team stops within six messages for the semantic approval reason.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Route Specialists with SelectorGroupChat

LAB 7

You expand OpsDesk into four specialist roles and run them through SelectorGroupChat. A custom selector function makes the learning route reproducible—planner to retriever to writer to reviewer—while the role descriptions, shared messages and combined termination show what a model-based selector would rely on in a dynamic workflow.

You'll build

.course_state/lab07-selector-team.json with four participant contracts, the observed route, approval evidence, message count, stop reason and bounded status.

Tools: SelectorGroupChat, selector_func, role descriptions, shared context, ExactTextTermination, MaxMessageTermination, replay clients

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 7 — Key Steps

1. Define non-overlapping planner, retriever, writer and reviewer roles.
2. Map the expected state transition in a custom selector function.
3. Create SelectorGroupChat with approval and eight-message limits.
4. Run the VPN workflow and inspect the selected route.
5. Compare fixed and selected patterns; retain bounded evidence.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 7 · VERIFY

Route Specialists with SelectorGroupChat

Test it

The route is user, planner, retriever, writer, reviewer; the reviewer approves the cited answer and the team stops within eight messages.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC 03 · HANDS-ON

Integrate and Verify the OpsDesk Workflow

LAB 8

You combine the stable retrieval questions, grounded RAG agent, round-robin review and selector workflow into one reproducible release check. You then run the full test suite, exercise missing-evidence and unsupported boundaries, inspect dependencies and tracked changes, name owners and pause triggers, and make an evidence-based guided-demo decision with a recovery path.

You'll build

.course_state/lab08-release-check.json plus a learner-authored operational record covering evidence, owners, pause triggers, last known-good identity and recovery for the complete OpsDesk workflow

Tools: Offline regression, retrieval cases, RAG trajectory, AgentChat teams, pytest, pip check, Git secret scan, operational readiness

© 2026 Tertiary Infotech Academy Pte Ltd

Lab 8 — Key Steps

1. Run the complete offline release check from stable source data.
2. Verify three retrieval cases, grounded RAG and both team patterns.
3. Exercise missing evidence, embedded instructions and hard boundaries.
4. Run tests, dependency health, tracked-change and secret checks.
5. Name owners, pause triggers and recovery; record READY or HOLD.

© 2026 Tertiary Infotech Academy Pte Ltd

LAB 8 · VERIFY

Integrate and Verify the OpsDesk Workflow

Test it

All offline checks are clean, sources and routes match expectations, aggregate usage stays within budget, no secret-shaped tracked value is found, and the readiness record names owners, pause triggers and a last known-good recovery path.

© 2026 Tertiary Infotech Academy Pte Ltd

TOPIC RECAP

Recap — Implementing a Multi-Agent AI Workflow

1. You can now: LO5 and LO6: define distinct writer and reviewer contracts, run a predictable RoundRobinGroupChat, and combine approval with a hard message limit

2. You can now: LO5 and LO6: configure distinct planner, retriever, writer and reviewer descriptions, control next-speaker routing, and verify a bounded selector workflow

3. You can now: LO6: run an end-to-end offline regression pack, inspect retrieval and team trajectories, check dependencies and secret hygiene, and record a bounded readiness or hold decision

© 2026 Tertiary Infotech Academy Pte Ltd

WRAP-UP

Course Summary & Next Steps

© 2026 Tertiary Infotech Academy Pte Ltd

LEARNING OUTCOMES

What You Achieved

1

Agent Foundations

Explain LLM agents, the AgentChat execution model, and the practical differences between AutoGen and CrewAI abstractions.

2

AgentChat in Practice

Configure and run an AutoGen AssistantAgent with a model client, clear messages, a typed read-only tool, and inspectable event evidence.

3

Retrieval Design

Design a Retrieval-Augmented Generation pipeline with source-labelled chunks, transparent ranking, metadata, context limits, and stable retrieval questions.

4

Grounded RAG

Build and verify a grounded RAG agent that cites retrieved evidence, treats documents as untrusted data, and handles insufficient evidence honestly.

5

Team Orchestration

Select and implement appropriate multi-agent patterns with specialist roles, explicit handoffs, shared context, and bounded speaker routing.

6

Operational Control

Apply termination, state management, guardrails, observability, regression checks, ownership, and recovery controls to an integrated AutoGen workflow.

© 2026 Tertiary Infotech Academy Pte Ltd

NEXT STEPS

From Classroom Workflow to Controlled Pilot

1. Keep the offline regression path as the reproducible baseline.
2. Change one model, retriever or routing variable at a time and compare evidence.
3. Add domain owners, access controls, telemetry and a rollback path before connecting live systems.
4. Rehearse missing evidence, forced limits and provider failure—not only the normal case.

© 2026 Tertiary Infotech Academy Pte Ltd

Keep Practising After the Course

1. Re-run the labs on your own machine — repetition is what makes the skills stick.
2. Apply the techniques to a real process or project in your own organisation.
3. Your course material stays available at https://lms-tms.tertiaryinfotech.com/.
4. Explore the follow-on courses at www.tertiarycourses.com.sg.

© 2026 Tertiary Infotech Academy Pte Ltd

THANK YOU FOR ATTENDING

Thank You!

You now have the practical skills from every lab in this course — keep applying them to your own work.

© 2026 Tertiary Infotech Academy Pte Ltd